

Пројектни задатак – Извјештај

Пројектни задатак представља дискретну симулацију AI агената који приступају одређеном виртуелном фајл систему. Иако, иницијално, замишљена као *multi-threaded* апликација, у финалној верзији је – због временских ограничења израђена као *single-threaded* апликација написана у **C++**, док сам за писање јединичних тестова користио **GTest** пакет.

Апликација се компајлира позивајући *Powershell* скрипата¹ која се налази на путањи `./scripts/build-gcc.ps1`. Оваква скрипта аутоматски позива **CMake** који компајлира апликацију и покреће јединичне тестове. Након што сви тестови прођу, скрипта чисти `/build` фолдер од свих „непотребних“ фајлова, остављајући само чисту извршну датотеку.

Апликација се покреће из командне линије, гдје јој се као додатни аргумент даје *JSON* датотека која садржи конфигурацију симулације за дати случај. *JSON* датотеке се налазе у директоријуму `/config/json/` док се датотеке потребне за извршење агената (листа операција и датотеке у виртуелном *file system-y*) налазе у директоријуму `/config/examples/<naziv_simulacije>`.

Конфигурациони фајл се парсира користећи *nlohmann/json* библиотеку и инстанцира се у објекат класе *Config*. Онда се за сваки агент и *mount* у конфигурационом фајлу инстанцирају објекти класе *AgentConfig* и *MountPoint* респективно.

Сам *VFS* се инстанцира уз помоћ класе *InMemoryVFS* која рекурзивно пролази кроз структуру директорија и мапира их у мапу чворова. Унутар *VFS-a* имплементиран је и *RW Lock* који омогућава *VFS-у* да „закључа“ приступ датотеци која је тренутно заузета од стране једног корисника (у овом случају агента) у случају да више агената желе да пишу на исту датотеку. Ако у том тренутку други агент покуша да закључа исту датотеку, операција се одбија уколико он има мањи приоритет (и ставља се на чекање). Уколико је његов приоритет већи, операција се дозвољава - претходном агенту се присилно

¹ Именица скрипта је *pluralia tantum* и као таква има само облик множине

одузима контрола као да је регуларно затворио датотеку док агент већег приоритета одмах преузима приступ.

Детекцијом застоја се бави класа *DeadlockGraph* која преко графа чекања, којим ограничава агент на само један ресурс. У случају да агент покуша закључати фајл који је већ заузет, симулација додаје грану у граф, те врши претрагу по дубини која тражи циклус. Уколико се пронађе циклус, операција се одбија, грана се брише и агент наставља даље. У супротном, агент се блокира и чека ослобађање ресурса. При сваком одбијању, исписује се комплетан пут циклуса.

Scheduler је имплементиран као приоритетни *scheduler* а преемпцијом. Он одржава листу слотова одређену максималним бројем агената из конфигурационе датотеке. На почетку сваког тика, планер пребацује све агенте чије је вријеме доласка = тиму у листу спремних агената, ослобађа слотове које држе заузети/блокирани агенти, провјерава да ли постоји агент са већим приоритетом од неког активног, и уколико има онда долази до преемпције.

На самом крају, слободни слотови се попуњавају агентима из реда према приоритету. Преемптовани агент не извршава операцију у тиму у коме је избачен.

За израду овог пројекта, у одређеним стварима ми је помогла вјештачка интелигенција. Примарно *inline* сугестије које предлаже *GitHub Copilot*, али сам имао значајну теоријску помоћ од стране *Claude Sonnet* модела. Примарно у разумијевању самог задатка и помоћи око архитектуре самог пројекта.

На самом крају пројекта, *Perplexity* сам користио као помоћ у рефакторизацији пројекта и генерисању примјер.